Dynamic Programming - 1

1. Maximum sum subarray - Kadance's Algorithm O(n)

Maintain two variable -
a. Max_till_index
b. Global_max

if
max_till_index > 0 then add the current element to previous sequence
else
start a new sequence from current_index.

update global_sequence_sum.

```
public int maxSubArray(int[] nums) {
    int n = nums.length;
    //Kadane's Solution - O(n)
    int max_so_far = nums[0], global_max = nums[0];

    for(int i = 1 ; i < n ; i++){

        if(max_so_far > 0)
            max_so_far += nums[i];
        else
            max_so_far = nums[i];

        if(max_so_far > global_max)
            global_max = max_so_far;

    }
    return global_max;
}
```

2. Max PRODUCT subarray - using KADANE's algo;

3 variables -
a. Max_so_far
b. Min_ending_here
c. Max_ending_here

```
public int maxProduct(int[] nums) {
        int maxSoFar = nums[0];
        int maxEndingHere = nums[0];
        int minEndingHere = nums[0];

        for (int i = 1; i < nums.length; i++) {
            int num = nums[i];
            int tempMax = Math.max(num, Math.max(maxEndingHere * num, minEndingHere * num));
            int tempMin = Math.min(num, Math.min(maxEndingHere * num, minEndingHere * num));

            maxEndingHere = tempMax;
            minEndingHere = tempMin;

            maxSoFar = Math.max(maxSoFar, maxEndingHere);
        }

        return maxSoFar;
    }
```

3. Delete and Earn Problem - 1D DP.| House Robber Problem

Pattern -> you have array elements where you **cannot** select subsequent elements.
if (choose ar[i])
Then (exclude ar[i -1] and ar[i + 1])

```
int[] dp = new int[maxValue + 1];
for(int n : nums)
  dp[n] += n;
int maxEarning = dp[1] ;
for(int i = 2 ; i <= maxValue ; i++ ){
  dp[i] = Math.max(dp[i-2] + dp[i], dp[i-1]);
  maxEarning = Math.max(maxEarning, dp[i]);
}
return maxEarning;
```

House Robber Problem - Similar

```
public int rob(int[] nums) {

    int rob1= 0, rob2 = 0;
    // [rob1, rob2, H3,   H4, H5, H6 ]        where H3 is decided to rob or not.
    // [X   , rob1, rob2, H4, H5, H6 ]        where H4 is decided to rob or not.
    for(int n  : nums){
      int temp = Math.max(n + rob1 , rob2 );
      rob1 = rob2;
      rob2 = temp;
    }
    return rob2;

  }
```

Now for **circular houses (House Robber II)**:

Because first and last are adjacent, you **can't take both**.

So the trick is:

- Case 1: Rob houses from index **0 to n-2** (exclude last)
- Case 2: Rob houses from index **1 to n-1** (exclude first)

Then answer is `max(case1, case2)`.

"First element affects last element" problems

Whenever choosing something at index `0` restricts index `n-1`, you split into two runs.

4. Coin Change Problem

Recursive + Memorisation
coinsRequired -> Global minimum coins required to represent sum S

TOP -DOWN Approach

```
class Solution {
    private Integer[][] memo;

    public int coinChange(int[] coins, int amount) {
        memo = new Integer[amount + 1][coins.length]; // Memoization table
        return dfs(coins, amount, 0);
    }

    private int dfs(int[] coins, int amount, int currentIndex) {

        if (amount == 0)
            return 0; // Found a combination, 0 coins need
        if (amount < 0 || currentIndex >= coins.length)
            return -1; // Impossible case (negative amount or all coins used)
        if (memo[amount][currentIndex] != null)
            return memo[amount][currentIndex];


        // Try using the current coin
        int resultUsingCurrent = dfs(coins, amount - coins[currentIndex], currentIndex);

        // Try skipping the current coin
        int resultSkippingCurrent = dfs(coins, amount, currentIndex + 1);

        // Find the minimum between using and skipping the current coin
        int minCoins = Math.min(resultUsingCurrent == -1 ? Integer.MAX_VALUE : resultUsingCurrent + 1,
                resultSkippingCurrent == -1 ? Integer.MAX_VALUE : resultSkippingCurrent);

        // Store the minimum for future reference
        memo[amount][currentIndex] = minCoins == Integer.MAX_VALUE ? -1 : minCoins;

   return memo[amount][currentIndex];
    }
}
```

Bottom - up Approach

```
class Solution {
    public int coinChange(int[] coins, int amount) {
        // Bottom up
        int[] dp = new int[amount+1];
        Arrays.fill(dp, Integer.MAX_VALUE);
        dp[0] = 0;
        for(int coin : coins){
            for(int value = coin ; value <= amount ; value++ )
                if(dp[value - coin] != Integer.MAX_VALUE) //dp[value - coin] + 1 does not exceeds MAX_Integer
                    dp[value] = Math.min(dp[value], dp[value - coin] + 1);
        }
        return dp[amount] == Integer.MAX_VALUE ? -1 : dp[amount];
    }
}
```

// with path - construction:

```
public static List<Integer> MinimumCoins(int n) {
    // Write your code here.
    int[] coins = {1,2,5,10,20,50,100,500,1000};
    int[] result = new int[n+1];
    int[] pick = new int[n+1];
    int INF = 1_000_000_000;
    Arrays.fill(result, INF);
    Arrays.fill(pick, -1);
    result[0] = 0;

    for(int c : coins){
        for(int i = c ; i <= n ; i++){
            if(result[i-c] != INF && result[i-c] + 1 < result[i]){
                result[i] = result[i-c] + 1;
                pick[i] = c;
            }
        }
    }


    List<Integer> answer = new ArrayList<>();

    if(result[n] == INF) return answer;

    //path construct
    int curr = n;
    while(curr > 0 ){
        int coin = pick[curr];
        answer.add(coin);
        curr = curr - coin;
    }

    return answer;
}
```

To do:

Fibonacci → larger number (10^18) -> cannot call for-loop → **Matrix Exponential technique**

1D DP Problems :

1. **Count Number of ways to reach Step N if starting from 0 and only 1 jump or 2 jump is allowed.**

This problem is same as Fibonacci series
The only difference is step 0 -> 1 but in fibonacci it's 0.

```
public static long countDistinctWayToClimbStair(int nStairs) {
        // Write your code here.
        int n = nStairs;

        if(n==0 || n == 1)
            return 1;

        int MOD = 1000000007;
        int prev1=1;
        int prev2=1;
```

```
        for(int i = 2; i <=n ;i++){
            int curr = (prev1 + prev2) % MOD;
            prev2 = prev1;
            prev1 = curr;
        }

        return prev1;
    }
```

2. **Frog Jump**

Recursive

```
class Solution {
    int minCost(int[] height) {
        // code here
        int n = height.length;
        if(n == 0)  return 0;

        return findCost(n-1, height);
    }

    int findCost(int currPos, int[] height){

        if(currPos == 1)
            return Math.abs(height[1] - height[0]);

        if(currPos == 2)
            return Math.min(Math.abs(height[2] - height[0]),
                            Math.abs(height[1] - height[0]) + Math.abs(height[2] - height[1]));

        int left = Math.abs(height[currPos] - height[currPos - 1]) + findCost(currPos - 1, height);

        int right = Math.abs(height[currPos] - height[currPos - 2]) + findCost(currPos - 2, height);

        return Math.min(left, right);
    }
}
```

Memorisation:

```
class Solution {
    int minCost(int[] height) {
        // code here
        int n = height.length;
        if(n == 0)  return 0;
        int dp[] = new int[n+1];
        Arrays.fill(dp, -1);
        dp[0]= 0;
        return findCost(n-1, height, dp);
    }

    int findCost(int currPos, int[] height, int[] dp){

        if(currPos == 0)
            return dp[0] = 0;

        if(dp[currPos] != -1)
            return dp[currPos];

        int left = Math.abs(height[currPos] - height[currPos - 1])
                    + findCost(currPos - 1, height, dp);

        int right = Integer.MAX_VALUE;
        if(currPos > 1) {
            right = Math.abs(height[currPos] - height[currPos - 2])
                    + findCost(currPos - 2, height, dp);
        }
        return dp[currPos] = Math.min(left, right);
    }
}
```

Iterative:

```
int minCost(int[] height) {
        // code here
        int n = height.length;
        if(n == 0 || n == 1)  return 0;
        if(n == 2)  return  Math.abs(height[1] - height[0]);

        int dp[] = new int[n];

        dp[0] = 0;
        dp[1] = Math.abs(height[1] - height[0]);
        for(int i = 2; i < n; i++){
            dp[i] = Math.min(Math.abs(height[i] - height[i-1]) + dp[i-1],
                             Math.abs( height[i] - height[i-2]) + dp[i-2]);
        }


        return dp[n-1];
    }
}

//Space -optimised
class Solution {
    int minCost(int[] height) {
        // code here
        int n = height.length;
        if(n == 0 || n == 1)  return 0;
        if(n == 2)  return  Math.abs(height[1] - height[0]);

        int prev2 = 0;
        int prev1 = Math.abs(height[1] - height[0]);
        for(int i = 2; i < n; i++){
            int curr = Math.min(Math.abs(height[i] - height[i-1]) + prev1,
                             Math.abs( height[i] - height[i-2]) + prev2);

            prev2 = prev1;
            prev1 = curr;
        }
        return prev1;
    }
}
```

3. **Max Sum without Adjacents**

Memorisation:

```java
class Solution {
    int findMaxSum(int arr[]) {
        // code here
        int N = arr.length;
        int[] dp = new int[N];
        Arrays.fill(dp, -1);

        return findSol(N-1, arr, dp);
    }

    int findSol(int P, int[] arr, int[] dp){
        if(P < 0 )
            return 0;

        if(dp[P] != -1)
            return dp[P];


        int include = arr[P] + findSol(P-2, arr, dp);
        int exclude = findSol(P-1, arr, dp);

        return dp[P] = Math.max(include, exclude);
    }
}
```

Space Optimised:

```java
//Solution - 1 : SC- O(N) , TC: O(N)
class Solution {
    int findMaxSum(int arr[]) {
        // code here
        int N = arr.length;

        if(N == 1)
            return arr[0];

        int[] dp = new int[N];
        Arrays.fill(dp, -1);
        dp[0] = arr[0];
        dp[1] = Math.max(arr[0],arr[1]);  // arr=[5, 2, 7]
                                  // here dp[0] = 5,also dp[1]= 5 (best answer possible till index 1)

        for(int i = 2 ; i < N ; i++){
            dp[i] = Math.max((arr[i] + dp[i-2]), dp[i-1]);
        }
        return Math.max(dp[N-1], dp[N-2]);
    }
}

//Solution - 2 : SC- O(1) , TC: O(N)
class Solution {
    int findMaxSum(int arr[]) {
        // code here
        int N = arr.length;

        if(N == 1)
            return arr[0];

        int prev2 = arr[0];
        int prev1 = Math.max(arr[0],arr[1]);

        for(int i = 2 ; i < N ; i++){
            int curr = Math.max((arr[i] + prev2), prev1);
            prev2 = prev1;
            prev1 = curr;
        }
        return Math.max(prev1, prev2);
    }
}
```

4. House Robber

```java
class Solution {
    public int findMaxSum(int arr[]) {
        // code here
        int N = arr.length;
        if(N==1)
            return arr[0];

        int prev2 = arr[0];
        int prev1 = Math.max(arr[0], arr[1]);

        for(int i = 2; i < N ; i++){
            int curr = Math.max(arr[i] + prev2, prev1);
            prev2 = prev1;
            prev1 = curr;
        }

        return prev1;
    }
}
```

Here if question says -> House_1 and House_N are connected (circular arr)

Then check solution → Max(`Solution(1,N), Solution(0,N-1)`)
Leave 1st element and get one solution
Leave last element and get one solution
Return max of both the above → result.

5. Geek's Training

arr[]= [[1, 2, 5], [3, 1, 1], [3, 3, 3]]

- There are `N` days.
- Each day you can do **one of 3 tasks** (let's call them `0, 1, 2`).
- `arr[day][task]` gives the points for doing that task on that day.
- **Constraint**: you cannot do the **same task on two consecutive days**.
- Goal → **maximize the total points** over all `N` days.

```java
class Solution {
    public int maximumPoints(int arr[][]) {
        // code here
        int N = arr.length;
        int[][] dp = new int[N][3];
        for(int[] t : dp)
            Arrays.fill(t, -1);

        int res = 0;

        for (int i = 0 ; i < 3; i++){
```

```
            dp[N-1][i] = arr[N-1][i] + getMaxProfit(arr, N-2, i, dp);
            if(dp[N-1][i] > res)
                res = dp[N-1][i];
        }
        // for(int[] t : dp)
        //     System.out.println(Arrays.toString(t));

        return res;
    }

    private int getMaxProfit(int arr[][], int pos, int lastTask, int[][] dp){
        if(pos < 0)
            return 0;

        if (dp[pos][lastTask] != -1)
            return dp[pos][lastTask];

        int res = 0;
        for(int i = 0 ; i < 3 ; i++){
            if(i != lastTask){
                int temp = arr[pos][i]  + getMaxProfit(arr, pos-1, i, dp);
                if(temp  > res)
                    res = temp;
            }
        }
        return dp[pos][lastTask] = res;   //store at return.
    }
}
```

Tabulation:

```
class Solution {

    public int maximumPoints(int arr[][]) {
        // code here
        int N = arr.length;
        int[][] dp = new int[N][3];

        //tabulation: bottom-up : small to large
        //base case : on day 0 if only task i is available.
        dp[0][0] = arr[0][0];
        dp[0][1] = arr[0][1];
        dp[0][2] = arr[0][2];

        for(int day = 1; day < N; day++){
            dp[day][0] = arr[day][0] + Math.max(dp[day-1][1], dp[day-1][2]); //for task0, select prev task1 or task2
            dp[day][1] = arr[day][1] + Math.max(dp[day-1][0], dp[day-1][2]);
            dp[day][2] = arr[day][2] + Math.max(dp[day-1][0], dp[day-1][1]);
        }

        return Math.max(dp[N-1][0], Math.max(dp[N-1][1], dp[N-1][2]));
    }
}
```

Approach 2:
// User function Template for Java

```
class Solution {

    public int maximumPoints(int arr[][]) {
        // code here
        int N = arr.length;
        int[][] dp = new int[N][4];

        //tabulation: bottom-up : small to large
        //base case : on day 0 if only task i is available.
        dp[0][0] = arr[0][0];
        dp[0][1] = arr[0][1];
        dp[0][2] = arr[0][2];

        dp[0][3] = Math.max(arr[0][0], Math.max(arr[0][1], arr[0][2]));

        //idea : dp[day][lastTask] = maximum points up to 'day' --> if yesterday's task was 'lastTask'

        for(int day = 1; day < N; day++){
            for(int last = 0; last < 4 ; last++){
                dp[day][last] = 0;
                for(int task = 0; task < 3 ; task++){
                    if(task != last){
                        int points = arr[day][task] + dp[day-1][task]; //dp[day-1][last] --> yesterday's choice was
anything except task.
                        dp[day][last] = Math.max(dp[day][task], points);
                    }
                }
            }

        }

        return dp[N-1][3]; // no restriction on last day
    }
}
```

Day 0-> dp[0] = [5, 5, 2, 5]

day = 1

Now arr[1] = [3, 1, 1]

We compute dp[1][last] for each possible last.

- dp[1][0] = max(arr[1][1] + dp[0][1], arr[1][2] + dp[0][2])//for task 0 -> prev task 1 and 2 are considered
  = max(1 + 5, 1 + 2) = max(6, 3) = 6
- dp[1][1] = max(arr[1][0] + dp[0][0], arr[1][2] + dp[0][2])
  = max(3 + 5, 1 + 2) = max(8, 3) = 8
- dp[1][2] = max(arr[1][0] + dp[0][0], arr[1][1] + dp[0][1])
  = max(3 + 5, 1 + 5) = max(8, 6) = 8
- dp[1][3] = max(arr[1][0] + dp[0][0], arr[1][1] + dp[0][1], arr[1][2] + dp[0][2])
  = max(3 + 5, 1 + 5, 1 + 2) = max(8, 6, 3) = 8

2D Matrix Problems

1. Explore all paths from (0,0) -> (M-1, N-1)

Recursive

```
class Solution {
    public int numberOfPaths(int m, int n) {
        // Code Here
        return countPaths(m-1,n-1,m,n);
    }

    private int countPaths(int x, int y, int m, int n){
        if(x <0 || y < 0)
            return 0;

        if(x == 0 && y == 0)
            return 1;

        int left = countPaths(x, y-1, m, n);
        int top = countPaths(x-1, y, m, n);
```

```
        return left + top;
    }
}

TC : 2 M * N
```

Memorisation:

```
class Solution {
    public int numberOfPaths(int m, int n) {
        // Code Here
        int[][] dp =  new int[m][n];
        for(int[] row : dp)
            Arrays.fill(row, -1);  //O(N + M)
        dp[0][0] = 1;
        return countPaths(m-1,n-1,m,n, dp);
    }

    private int countPaths(int x, int y, int m, int n, int[][] dp){
        if(x <0 || y < 0)
            return 0;
        if(dp[x][y] != -1)
            return dp[x][y];
        int left = countPaths(x, y-1, m, n, dp);
        int top = countPaths(x-1, y, m, n, dp);
        return dp[x][y] = left + top;
    }
}
```

TC: O (M * N)
SC: O (M*N) + O(N + M)

Tabulation:

```
class Solution {
    public int numberOfPaths(int m, int n) {
        // Code Here
        int[][] dp =  new int[m][n];
        //Tabulation : Bottom up (smaller cases to larger cases)
        //base case declaration
        dp[0][0] = 1;

        //copy recurrence
        for(int i = 0; i < m ; i++){
            for(int j = 0; j < n ; j++){
                if(i == 0 && j == 0)
                    dp[i][j] = 0;
                else {
                    int up = 0;
                    if(i > 0) up = dp[i-1][j];

                    int left = 0;
                    if(j > 0) left = dp[i][j-1];

                    dp[i][j] = left + right;
                }

            }
        }
        return countPaths(m-1,n-1,m,n, dp);
    }
}
```

SC : O(N*M)
TC: O(N*M)

If there is prev row and prev column used in dp answer formation → ***it can be space optimised***

For loop -> moves from left to right and top to bottom.
We don't need the full 2D array, rather we can create a 1D array.

```
class Solution {
    public int numberOfPaths(int m, int n) {
        // Code Here
        int[] prev  = new int[n];

        Arrays.fill(prev, 0);

        for(int i = 0; i < m; i++){
            int[] curr = new int[n];

            for(int j = 0; j < n ; j++){
                if(i == 0 && j == 0)
                    curr[j] = 1;
                else {
                    int up = 0;
                    int left = 0;
                    if(i > 0) up = prev[j];
                    if(j > 0) left = curr[j-1];

                    curr[j] = up + left;
                }
            }
            prev = curr;
        }
        return prev[n-1];
    }
}
```

//BEST solution : GOOGLE (using combinations)

```
class Solution {
    public int numberOfPaths(int m, int n) {
        // Code Here
        //MOST optimal using combination;

        // if M, N, last row
        //from (0,0) to move to (M,N)
        // we need (M-1) left moves  + (N-1) Bottom moves
        int N = m + n - 2;
        int r = m -1;

        double res = 1;
        for(int i = 1; i <= r ; i++){
            res = res * (N - r + i)/i;
        }

        return (int)res;
```

```
    }

}
```

2. **Unique Paths with Obstacles**

if(arr[i][j] == -1) → dead-end

Memorisation

```
import java.util.*;

public class Solution {
    static int mazeObstacles(int n, int m, ArrayList<ArrayList<Integer>> mat) {
        // Write your code here.
        int[][] dp = new int[n][m];
        for(int[] row : dp)
            Arrays.fill(row, -1);
        return findPaths(n-1, m-1, mat, dp);
    }

    static int findPaths(int x, int y, ArrayList<ArrayList<Integer>> mat, int[][] dp){

        if(x < 0 || y < 0)
            return 0;

        if(x == 0 && y == 0)
            return dp[0][0] = 1;

        if(dp[x][y] != -1)
            return dp[x][y];

        if(mat.get(x).get(y) == -1)
            return dp[x][y] = 0;

        int top = findPaths(x-1, y, mat, dp);
        int left = findPaths(x, y-1, mat, dp);

        return dp[x][y] = top + left;
    }
}
```

Tabulation:

```
import java.util.*;

public class Solution {
    static int mazeObstacles(int n, int m, ArrayList<ArrayList<Integer>> mat) {
        // Write your code here.
        int[][] dp = new int[n][m];
        int MOD = 1_000_000_007;

        for(int i = 0 ; i < n ; i++){
            for(int j = 0 ; j < m ; j++){

                if(mat.get(i).get(j) == -1){
                    dp[i][j] = 0;
                    continue;
                }

                if(i == 0 && j == 0){
                    dp[i][j] = 1;
                    continue;
                }
                long up = 0;
                if(i > 0)   up = dp[i-1][j];

                long left = 0;
                if(j > 0) left = dp[i][j-1];

                long paths = (up + left) % MOD;


                dp[i][j] = (int)paths;

            }
        }
        return dp[n-1][m-1];
    }
}
```

Space Optimised:

```
import java.util.*;

public class Solution {
    static int mazeObstacles(int n, int m, ArrayList<ArrayList<Integer>> mat) {
        // Write your code here.

        int MOD = 1_000_000_007;
        //space-optimised
        // replace dp[i-1] -> prev, dp[i] -> curr.


        int[] prev = new int[m];

        for(int i = 0 ; i < n ; i++){
            int[] curr = new int[m];
            for(int j = 0 ; j < m ; j++){

                if(mat.get(i).get(j) == -1){
                    curr[j] = 0;
                    continue;
                }

                if(i == 0 && j == 0){
                    curr[j] = 1;
                    continue;
                }
                long up = 0;
                if(i > 0)   up = prev[j];

                long left = 0;
                if(j > 0) left = curr[j-1];

                long paths = (up + left) % MOD;
```

```
                curr[j] = (int)paths;

            }
            prev = curr;
        }
        return prev[m-1];
    }

}
```

3. Min sum path in 2D grid

Recursive

```
import java.util.* ;
import java.io.*;
public class Solution {
    public static int minSumPath(int[][] grid) {
        // Write your code here.
        int N = grid.length;
        int M = grid[0].length;
        return findMinPath(N-1, M-1, grid);
    }

    public static int findMinPath(int x, int y, int[][] grid){
        if(x < 0 || y < 0)
            return Integer.MAX_VALUE;

        if(x == 0 && y == 0)
            return grid[0][0];

        long left = findMinPath(x-1, y, grid);
        long top = findMinPath(x, y - 1, grid);

        return grid[x][y] + (int)Math.min(left, top);
    }
}
```

Memorisation:

```
import java.util.* ;
import java.io.*;
public class Solution {

    public static int minSumPath(int[][] grid) {
        // Write your code here.
        int N = grid.length;
        int M = grid[0].length;
        int[][] dp = new int[N][M];
        for(int[] row: dp)
            Arrays.fill(row, -1);

        return findMinPath(N-1, M-1, grid, dp);
    }

    public static int findMinPath(int x, int y, int[][] grid, int[][] dp){
        if(x < 0 || y < 0)
            return 1_000_000_007;

        if(x == 0 && y == 0)
            return dp[x][y] = grid[x][y];

        if(dp[x][y] != -1)
            return dp[x][y];

        int left = findMinPath(x-1, y, grid, dp);
        int top = findMinPath(x, y - 1, grid, dp);

        return dp[x][y] = grid[x][y] + Math.min(left, top);
    }
}
```

Tabulation + Space Optimised

```
import java.util.* ;
import java.io.*;
public class Solution {

    public static int minSumPath(int[][] grid) {
        // Write your code here.
        int N = grid.length;
        int M = grid[0].length;
        int[] prev = new int[M];
        int INF = 1_000_000_007;

        for(int i = 0 ; i < N ; i++){
            int[] curr = new int[M];
            for(int j = 0 ; j < M ; j++){
                if(i == 0 && j == 0){
                    curr[j] = grid[i][j];
                    continue;
                }
                int top = INF;
                if(i > 0) top = prev[j];

                int left = INF;
                if(j > 0) left = curr[j-1];

                curr[j] = grid[i][j] + Math.min(top, left);
            }
            prev = curr;
        }
        // System.out.println(Arrays.toString(prev));
        return prev[M-1];
    }
}
```

4. Triangle Traversal

For an triangle array, from (0,0) reach any node in last row in minimum cost.

Here start from a single fixed point (0,0) and the final answer can reach any **node in the last row.**

```
import java.util.* ;
import java.io.*;
public class Solution {
    public static int minimumPathSum(int[][] triangle, int n) {
        // Write your code here.
```

```
        if(n == 1)
            return triangle[0][0];

        int[][] dp = new int[n][n];
        for(int[] row : dp)
            Arrays.fill(row, -1);

        return dp[0][0] = triangle[0][0] + Math.min(findMinPath(1,0, triangle, n, dp), findMinPath(1,1,
triangle, n,dp));
    }

    private static int findMinPath(int x, int y, int[][] triangle, int n,int[][] dp){

        if(x < 0 || y < 0 || x == n)
            return 0;

        if(dp[x][y] != -1)
            return dp[x][y];

        int path1 = triangle[x][y]  + findMinPath(x+1, y, triangle, n, dp);
        int path2 = triangle [x][y] + findMinPath(x+1, y+1, triangle, n, dp);

        return dp[x][y] = Math.min(path1, path2);
    }
}
```

Tabulation:

```
import java.util.* ;
import java.io.*;
public class Solution {
    public static int minimumPathSum(int[][] triangle, int n) {
        // Write your code here.
        if(n == 1)
            return triangle[0][0];

        int[][] dp = new int[n][n];
        for(int[] row : dp)
            Arrays.fill(row, Integer.MAX_VALUE);

        for(int i = 0 ; i < n ; i++){
            for(int j = 0 ; j <= i; j++){
                if(i == 0 && j == 0)
                    dp[i][j] = triangle[i][j];
                else{
                    // can reach [i][j] from [i-1] [j] and [i-1][j-1]
                    int option1 = dp[i-1][j]
                    int option2 = Integer.MAX_VALUE;
                    if(j > 0)
                        option2 = dp[i-1][j-1];

                    dp[i][j] = triangle[i][j] + Math.min(option1, option2);
                }
            }
        }

        int result = dp[n-1][0];
        for(int idx = 1; idx < n ; idx++)
        {
            if(result > dp[n-1][idx])
                result = dp[n-1][idx];
        }
        return result;
    }
}
```

Space optimised:

```
public class Solution {
    public static int minimumPathSum(int[][] triangle, int n) {
        // Write your code here.
        if(n == 1)
            return triangle[0][0];
        int[] prev = new int[n];
        for(int j = 0 ; j < n ; j++)    prev[j] = triangle[n-1][j];

        for(int i = n-2 ; i >=0 ; i--){
            int[] curr = new int[n];
            for(int j = i; j >=0 ; j--){
                int option1 = triangle[i][j] + prev[j];
                int option2 = triangle[i][j] + prev[j+1];
                curr[j] = Math.min(option1, option2);
            }
            System.out.println(Arrays.toString(prev));
            prev = curr;
        }

        return prev[0];
    }
}
```

5. Maximum Sum path from any node from first row to any node in last row

Recursive + dp

```
public class Solution {
    public static int getMaxPathSum(int[][] matrix) {
        // Write your code here

        int N = matrix.length;
        int M = matrix[0].length;
        int INF = -1_000_000_000;
        int res = INF;

        int[][] dp = new int[N][M];
        for(int[] row : dp)
            Arrays.fill(row, INF);
       // check from each node
        for(int i= 0; i < M ; i++){
            int pathSum = findMaxPath(N-1, i, matrix, dp);
            if(pathSum > res)
                res = pathSum;
        }

        return res;

    }
```

```java
    private static int findMaxPath(int x, int y, int[][] matrix, int[][] dp){
        int N = matrix.length;
        int M = matrix[0].length;
        int INF = -1_000_000_000;

        if(x < 0 || y < 0 || y > M-1)
            return INF;

        if(dp[x][y] != INF)
            return dp[x][y];

        if(x == 0)
            return dp[x][y] = matrix[x][y];

        int top = findMaxPath(x-1, y, matrix, dp);
        int left = findMaxPath(x-1, y-1, matrix, dp);
        int right = findMaxPath(x-1, y+1, matrix, dp);

        return dp[x][y] = matrix[x][y] + Math.max(top, Math.max(left, right));
    }
}
```

Tabulation

```java
public class Solution {
    public static int getMaxPathSum(int[][] matrix) {
        // Write your code here

        int N = matrix.length;
        int M = matrix[0].length;
        int INF = -1_000_000_000;

        int[][] dp = new int[N][M];
        for(int i = 0 ; i < N ; i++){
            for(int j = 0; j < M ; j++){
                if(i == 0){
                    dp[i][j] = matrix[i][j];
                } else {
                    int top = dp[i-1][j];
                    int dLeft = INF;
                    if(j > 0)   dLeft = dp[i-1][j-1];
                    int dRight = INF;
                    if(j < M -1) dRight =dp[i-1][j+1];

                    dp[i][j] = matrix[i][j] + Math.max(top, Math.max(dLeft, dRight));
                }
            }
        }

        int res = dp[N-1][0];
        for(int k = 1 ; k < M ; k++)
            res = Math.max(res, dp[N-1][k]);

        return res;
    }
}
```

//Space optimised

```java
public class Solution {
    public static int getMaxPathSum(int[][] matrix) {
        // Write your code here

        int N = matrix.length;
        int M = matrix[0].length;
        int INF = -1_000_000_000;

        //space-optimisation
        int[] prev = new int[M];
        for(int i = 0 ; i < N ; i++){
            int[] curr = new int[M];
            for(int j = 0; j < M ; j++){
                if(i == 0){
                    curr[j] = matrix[i][j];
                } else {
                    int top = prev[j];
                    int dLeft = INF;
                    if(j > 0)   dLeft = prev[j-1];
                    int dRight = INF;
                    if(j < M -1) dRight = prev[j+1];

                    curr[j] = matrix[i][j] + Math.max(top, Math.max(dLeft, dRight));
                }
            }
            prev = curr;
        }
        int res = prev[0];
        for(int k = 1 ; k < M ; k++)
            res = Math.max(res, prev[k]);

        return res;
    }
}
```

3-D DP Problem

1. **Cherry Pick -1**

   Here not only value, but also the position where I am also has to be taken care of.
   Consider 2 Persons P1(x1,y1) and P2(x2,y2) starts from (0,0) and their goal is to reach (N-1, N-1)

   At each step there are 4 possible options.

   So the **4 possibilities** (Person1 down/right, Person2 down/right) are:

   1. P1 ↓(down), P2 ↓(down) : `dfs(x1+1, y1, x2+1)`
   2. P1 ↓(down), P2(right) → : `dfs(x1+1, y1, x2)`
   3. P1(right) →, P2 ↓(down) : `dfs(x1, y1+1, x2+1)`
   4. P1(right) →, P2(right) → : `dfs(x1, y1+1, x2)`

   `Here observe (x1 + y1) == (x2 + y2) for each step. (you either move both or none in recursion)`

```java
class Solution {
    public int cherryPickup(int[][] grid) {

        int ans = dfs(grid, 0, 0, 0);
        return Math.max(0, ans);

    }
```

```java
    int dfs(int[][] grid, int x1, int y1, int x2){
        int y2 = (x1 + y1) - x2; //since x1+y1 == x2+y2 always (each moves one step at a time either in x-axis or y-axis,
                                 // so relative change is position is always 1 unit from initial position)

        int N = grid.length; //N*N matrix
        if(x1 >= N || y1 >= N || x2 >= N || y2 >= N ){
            return Integer.MIN_VALUE;   //invalid
        }

        if(grid[x1][y1] == -1 || grid[x2][y2] == -1) //OR condition
            return Integer.MIN_VALUE; //thorn you either move both or none

        if(x1 == N-1 && y1 == N-1)
            return grid[x1][y1];    //when x1,y1 == (N-1, N-1) then x2,y2 has to be (N-1,N-1)
                                    // since (x1+y1) == (x2+y2)

        int cherry;
        if(x1 == x2 && y1 == y2)
            cherry = grid[x1][y1];  //same cell -> include once only
        else
            cherry = grid[x1][y1] + grid[x2][y2];
        int bestPath = Math.max(
                Math.max(dfs(grid, x1+1, y1, x2+1), dfs(grid, x1+1, y1, x2)),
                Math.max(dfs(grid, x1, y1+1, x2+1), dfs(grid, x1, y1+1, x2)));

        return cherry + bestPath;

    }
}
```

Memorised

```java
class Solution {
    private int dfs(int[][] grid, int r1, int c1, int r2, Integer[][][] dp) {
        //Problem reduces to 2 people starting from 0,0 & reaching at m-1, n-1, instead of back-n-forth
        //r1+c1 = r2+c2, as for each step, 1 value is added to either r or c, so total r+c remains the same
        int c2 = r1 + c1 - r2, n = grid.length;

        if (r1 >= n || c1 >= n || r2 >= n || c2 >= n || r1 > r2 || grid[r1][c1] == -1 || grid[r2][c2] == -1)
            return Integer.MIN_VALUE; // Bottom (1 Down) and Right (1 Right) of the grid causes 2 Negatives
                                      // Blockade can cause all 4 Negatives, ultimately 0 = max(0, -) is returned

        if (r1 == n - 1 && c1 == n - 1)
            return grid[r1][c1];

        if (dp[r1][c1][r2] != null)
            return dp[r1][c1][r2];

        //any grid (i,j) can be visited in exactly i+j steps, so both persons will arrive at the same cell together but not
        //one after the other
        return dp[r1][c1][r2] = grid[r1][c1] + (r1 == r2 && c1 == c2 ? 0 : grid[r2][c2]) +
            Math.max(
                Math.max(dfs(grid, r1 + 1, c1, r2 + 1, dp), dfs(grid, r1, c1 + 1, r2 + 1, dp)),
                Math.max(dfs(grid, r1 + 1, c1, r2, dp), dfs(grid, r1, c1 + 1, r2, dp))
            );
    }

    public int cherryPickup(int[][] grid) {
        return Math.max(0, dfs(grid, 0, 0, 0, new Integer[grid.length][grid.length][grid.length]));
    }
}
```

2. **Cherry Pickup-2**

Here 2 bots are there, one at (0,0) and other at (0, M-1) → goal to reach the last row with max cherries.

Memorisation

```java
class Solution {
    public int cherryPickup(int[][] grid) {
        int N = grid.length, M = grid[0].length;
        Integer[][][] dp = new Integer[N][M][M];
        return Math.max(0, dfs(grid, 0, 0, M - 1, dp, N, M)); // ensure non-negative
    }

    int dfs(int[][] grid, int x, int y1, int y2, Integer[][][] dp, int N, int M) {
        // Out of bounds → invalid
        if (x >= N || y1 < 0 || y2 < 0 || y1 >= M || y2 >= M)
            return Integer.MIN_VALUE;

        // Obstacle → invalid
        if (grid[x][y1] == -1 || grid[x][y2] == -1)
            return Integer.MIN_VALUE;

        // Reached bottom row
        if (x == N - 1) {
            return (y1 == y2) ? grid[x][y1] : grid[x][y1] + grid[x][y2];
        }

        if (dp[x][y1][y2] != null) return dp[x][y1][y2];

        int maxPath = Integer.MIN_VALUE;
        int[] change = {-1, 0, 1};

        for (int d1 : change) {
            for (int d2 : change) {
                int next = dfs(grid, x + 1, y1 + d1, y2 + d2, dp, N, M);
                if (next != Integer.MIN_VALUE) {
                    int value = (y1 == y2) ? grid[x][y1] : grid[x][y1] + grid[x][y2];
                    maxPath = Math.max(maxPath, value + next);
                }
            }
        }

        return dp[x][y1][y2] = maxPath;
    }
}
```

Tabulation:

Pending

Dynamic Programming - 2 (on Subsequence)

1. **Candy Distribution Problem - LC 135**

Q . There are **n children** standing in a line. Each child is assigned a rating value given in the integer array ratings. You are giving candies to these children subjected to the following requirements:

- Each child must have at least one candy.
- Children with a higher rating get more candies than their neighbors.

Return the minimum number of candies you need to have to distribute the candies to the children.

2-way DP: when value at a position **arr[i] depends on values from both the neighbours** then it needs 2 traversal
Left to right -> following a condition
And Right to left -> following a condition

```
class Solution {
    public int candy(int[] ratings) {

        int[] candies = new int[ratings.length];
        Arrays.fill(candies, 1);

        //left to right candy distribution condition
        for(int i= 1; i < ratings.length ; i++){
            if(ratings[i] > ratings[i-1])
                candies[i] = candies[i-1] + 1;
        }

        //right to left candy distribution condition

        // System.out.println("left :" +  Arrays.toString(candies));

        for(int i = ratings.length - 2; i >=0 ; i--){
            if(ratings[i] > ratings[i+1])
                candies[i] = Math.max(candies[i], candies[i+1] + 1);
        }

        // System.out.println("right :"+ Arrays.toString(candies));

        int sum = 0;
        for(int c : candies)    sum += c;

        return sum;

    }
}
```

DP - enhanced recursion:

Recursion -> Memorisation -> Tabulate

Hint:
- finding the optimal solution
- some choice is given to include an element in the answer.
- Overlapping calls. (If you identify 2 function calls in the solution, it might be dp → calling function with smaller value than parent.)

Parent Questions:
1. 0-1 knapsack (6 variations)
2. Unbounded Knapsack(5)
3. Fibonacci (7)
4. LCS (15)
5. LIS (10)
6. Kadane's Algo (6)
7. Matrix Chain Multiplication (7)
8. DP on trees (4)
9. DP on Grid (14)
10. Others Miscellaneous (5)

❖ Parent : 0-1 Knapsack Problem :
- ➢ Subset Sum equals K
- ➢ Equal Sum Partition
- ➢ Count of Subset Sum
- ➢ Minimum Subset Sum Difference
- ➢ Target Sum
- ➢ Number of subset - given difference.

Fractional Knapsack: can break items into smaller weights.
0/1 knapsack: allowed to add a full item **only once**.
Unbounded Knapsack: allowed to add a full item, **with multiple occurrences.**

❖ 0/1 Knapsack :

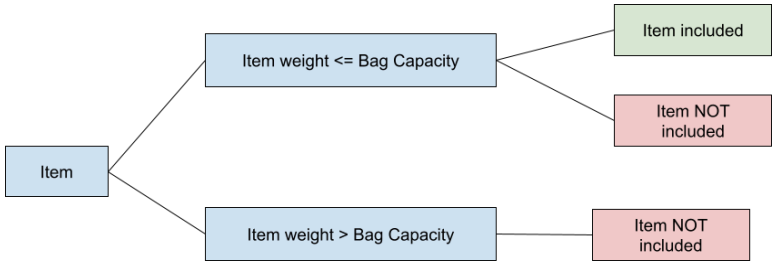
Identification :

Weights	1	3	4	5	8
Value	1	4	5	7	3

Finding -> **Maximum** Profit if Weight capacity of bag is say 10kg

Hints:
Maximum/Minimum/ Smallest/ Largest → **Optimal** Solution
Every item has the 2 **choices** -> get included in the final answer or left out.

Choice Diagram :



Base Condition:
Think of the smallest valid input -> Return the expected answer.
In this case : smallest input for item -> weight 0 kg -> item value return 0.

Recursive:

```
class Solution {
    // Function to return max value that can be put in the knapsack of capacity.
    static int knapSack(int capacity, int val[], int wt[]) {
        // code here
        return dp(capacity, val, wt, val.length - 1);
    }

    static int dp(int capacity, int val[], int wt[], int currIdx){
        //base condition
        if(currIdx == -1 || capacity == 0)
            return 0;

        if(wt[currIdx] <= capacity)
            return Math.max(
                    val[currIdx] + dp(capacity - wt[currIdx], val, wt, currIdx - 1),   //item included
                    dp(capacity, val, wt, currIdx - 1));  //item NOT included
        else{
            //item weight > bag capacity;
            return dp(capacity, val, wt, currIdx - 1);  //item NOT included
        }
```

```
    }
}
```

TC: O(2^n)
SC:O(n)

Memorisation:

```
public class Solution {

    // dp[weight][value]
    static int[][] dp;

    static int knapsack(int[] weight, int[] value, int n, int maxWeight) {
            dp = new int[n+1][maxWeight + 1];

            for(int[] arr : dp)
                Arrays.fill(arr, -1);

            return recursion(weight, value, n, maxWeight);
    }

    static int recursion(int[] weight, int[] value, int n, int maxWeight){

        if(dp[n][maxWeight] != -1)
            return dp[n][maxWeight];

        if(n == 0 || maxWeight == 0)
            return dp[n][maxWeight] = 0;

        if(weight[n-1] > maxWeight)
            return dp[n][maxWeight] = recursion(weight, value, n-1, maxWeight);

        return dp[n][maxWeight] = Math.max(
                value[n-1] + recursion(weight, value, n-1, maxWeight - weight[n-1]),
                recursion(weight, value, n-1, maxWeight));
    }
}
```

0/1 knapsack: Tabulation

```
static int knapsack(int[] weight, int[] value, int n, int maxWeight) {

        // tabulation
        int[][] dp = new int[n+1][maxWeight + 1];

        for(int[] arr : dp)
            Arrays.fill(arr, -1);

        for(int i = 0; i < n + 1; i++) //n
            for(int j = 0 ; j < maxWeight + 1; j++) //maxWeight
            {
                if(i == 0 || j == 0)
                    dp[i][j] = 0;
                else
                {
                    if(weight[i-1] <= j)
                        dp[i][j] = Math.max(dp[i-1][j], value[i-1] + dp[i-1][j - weight[i-1]]);
                                    //since we cannot reuse the element so picking from i-1 index
                    else{
                        //weight greater than capacity
                        dp[i][j] = dp[i-1][j];
                    }

                }


            }

        return dp[n][maxWeight];
    }
```

Space Optimised:

```
class Solution {
    static int knapsack(int W, int val[], int wt[]) {
        // code here
        int N = val.length;
        int[] prev = new int[W+1];

        for(int i = 1; i <= N; i++){
            int value = val[i-1];
            int weight = wt[i-1];
            int[] curr = new int[W+1];

            for(int capacity = 1; capacity <= W; capacity++){
                int take = 0;
                if(weight <= capacity)
                    take = value + prev[capacity - weight];

                int notTake = prev[capacity];

                curr[capacity] = Math.max(take, notTake);
            }
            prev = curr;
        }
        return prev[W];
    }
}
```

Backward approach (without rolling curr arrays)

```
class Solution {
    static int knapsack(int W, int val[], int wt[]) {
        int N = val.length;
        int[] dp = new int[W + 1];  // dp[capacity] = max value for given capacity

        for (int i = 0; i < N; i++) {
            int weight = wt[i];
            int value = val[i];

            // iterate backwards so we don't overwrite needed states → 0/1 knapsack
            for (int capacity = W; capacity >= weight; capacity--) {
                dp[capacity] = Math.max(dp[capacity], value + dp[capacity - weight]);
            }
        }
        return dp[W];
    }
}
```

First item (val=2, wt=2)

Loop: `for (capacity = 5.. 2)`

- capacity=5 → max(0, 2 + dp[3]=0) = 2
- capacity=4 → max(0, 2 + dp[2]=0) = 2
- capacity=3 → max(0, 2 + dp[1]=0) = 2
- capacity=2 → max(0, 2 + dp[0]=0) = 2

❖ *Subset Sum equals K :*

Question: Given an array of Integers arr = [5,4,3,2,1] return if **subset sum of K** is possible considering you can select the item only once.

Knapsack variation**.**

Boolean[] can hold false, true and null which is not supported in boolean[];

Boolean[] dp = new Boolean[2]; → stores ***null*** by default.
boolean[] dp = new boolean[2]; → stores ***false*** by default.

```
public class Solution {
    static Boolean[][] dp;
    public static boolean subsetSumToK(int n, int k, int arr[]){
        // Write your code here.
        dp = new Boolean[n+1][k+1];
        return checkSubsetSum(arr, n, k);
    }

    static Boolean checkSubsetSum(int[] arr, int n, int target) {
        // Base Case: If target is 0, a subset exists
        if (target == 0) return true;

        // Base Case: If no items are left, no subset can form the target
        if (n == 0) return false;

        // Check memoized result
        if (dp[n][target] != null)
              return dp[n][target];

        // Exclude the current item
        Boolean exclude = checkSubsetSum(arr, n - 1, target);
        // Include the current item if it is less than or equal to the target
        Boolean include = false;
        if (value[n - 1] <= target) {
            include = checkSubsetSum(arr, n - 1, target - arr[n - 1]);
        }
        // Memoize and return the result
        return dp[n][target] = exclude || include;
    }

}
```

Bottom-up: Tabularisation

Arr = [4, 3, 2, 1] sum= 5

Sum (->) Item (below)	0	1	2	3	4	5
0:[]	True	False	False	False	False	False
1: [4]	True					
2: [4,3]	True					
3: [4,3,2]	True					
4: [4,3,2,1]	True					

Sub-problem: **dp[0][0]** : given an empty arr[] can we create a sum of **0** -> **True** (empty arr {}**)**
dp[1][0]: given an arr: [4] can we create a sum of **0** -> **True** (empty arr {}**)**
dp[2][0]: given an arr: [4,3] can we create a sum of **0** -> **True** (empty arr {}**)**

dp[0][1]: given an empty arr[] can we create a sum of **1** : **False**
dp[0][2]: given an empty arr[] can we create a sum of **2** : **False**

```
public static boolean subsetSumToK(int n, int k, int arr[]){
        // Write your code here.
        boolean dp[][] = new boolean[n+1][k+1];

        //given any arr, we can create 0 sum subset -> empty set.
        for(int i = 0; i <=n ; i++)   dp[i][0] = true;

        for(int i=1; i <= N; i++){
            for(int target = 1; target <= k ; target++){
                boolean include = false;

                if(arr[i-1] <= target)  //check if added value is greater than capacity.
                    include = dp[i-1][target - arr[i-1]];

                boolean exclude = dp[i-1][target];
                dp[i][target] = include || exclude;
            }
        }
        return dp[n][k];
    }
```

Space Optimised

```
static Boolean isSubsetSum(int arr[], int sum) {
        // code here
        int N = arr.length;
        boolean[] prev = new boolean[sum+1];

        prev[0] = true;

        for(int i=1; i <= N; i++){
            boolean[] curr = new boolean[sum+1];
            curr[0] = true;     //always can form an empty set.
            for(int target = 1; target <= sum ; target++){
                boolean include = false;

                if(arr[i-1] <= target)
                    include = prev[target - arr[i-1]];

                boolean exclude = prev[target];
                curr[target] = include || exclude;
            }
            prev = curr;
```

```
        }
        return prev[sum];
    }
```

❖ *Equal Sum Partition : [LC - 416]*

Question: Given an array of integers arr [1,5,10,6] find if it can be **divided into 2 subsets of equal sum.**

Solution: 0/1 knapsack variation -> with sum = sum(Arr) / 2;
Here K = 22/2 = 11;

	0	1	2	3	4	5	6	7	8	9	10	11
0	True	F	F	F	F	F	F	F	F	F	F	F
1	True	True	F	F	F	F	F	F	F	F	F	F
5	True	True	F	F	F	True	True	F	F	F	F	F
10	True	True	F	F	F	F	F	F	F	F	True	True
6	True	True	F	F	F	True	True	True	F	F	True	True

```
public boolean equalSumPartition(int[] nums) {
        int sum = 0;
        for(int num : nums)
            sum += num;

       //if the sum is odd then answer -> False.
        if(sum % 2 !=0 )    return false;

        int reqSum = sum/2;
        int n = nums.length;
        boolean[][] dp = new boolean[n + 1][reqSum + 1];

        //base case -> empty set with 0 sum always possible.
        for(int i = 0; i <= n ; i++)    dp[i][0] = true;

        for(int i = 1 ; i <= n; i++){
            for(int target = 1; target <= reqSum ; target++){

                if(nums[i-1] > target)
                    dp[i][target] = dp[i-1][target];

                else
                    dp[i][target] = dp[i-1][target] || dp[i-1][target - nums[i-1]];

            }
        }
        return dp[n][reqSum];
    }
```

❖ *Partition array into 2 subsets such that difference of sum is minimum*

Consider array : [2,4,3] Partition it into two subsets such that the **difference** between sum of two subsets S_1 and S_2 is **minimum**.

Let Total Sum of arr[] is → **T,**

We want to minimise → $\mathbf{S_2 - S_1}$ considering $S_2 > S_1$
$S_2 = T - S_1$
⇒ $\mathbf{S_2 - S_1}$ ⇒ $\mathbf{(T - S_1) - S_1}$
⇒ $\mathbf{T - 2S_1}$ → **Final Equation to minimize.**

Last row for dp[] arr for subset sum gives all possible values of subset sum possible. We can use the equation to find the minimum value considering

$\mathbf{S_1}$ **= Range (0, T/2)**

arr[] = {2, 4, 3}
T = 9

	0	1	2	3	4	5	6	7	8	9
{}	T	F	F	F	F	F	F	F	F	F
2	T	F	T	F	F	F	F	F	F	F
4	T	F	T	F	T	F	T	F	F	F
3	T	F	T	T	T	T	T	T	F	T

$\mathbf{S_1}$ **Range (0, T/2) ⇒ (0, 9/2) ⇒ (0,4)**

S1	$T - 2S_1$	S_1	S_2
2	9 - 2(2) = 5	{ 2 }	{ 4,3 }
3	9 - 2(3) = 3	{ 3 }	{ 2,4 }
4	9 - 2(4) = 1 (Minimum)	{ 4 }	{ 2,3 }

```
public static int minSubsetSumDifference(int[] arr, int n) {

        int totalSum = 0;
        for(int x : arr)   totalSum += Math.abs(x);

        boolean dp[][] = new boolean[n + 1][totalSum + 1];

        for(int i = 0 ; i < n + 1; i++) dp[i][0] = true;

        for(int i = 1 ; i <= n; i++){
            for(int target = 1; target <= totalSum ; target++){

                if(arr[i-1] > target)
                    dp[i][target] = dp[i-1][target];
                else
                    dp[i][target] = dp[i-1][target] || dp[i-1][target - arr[i-1]];
            }
        }

        // S1 = TotalSum/2
        //  Min(TotalSum - 2S1 ) => ans.

        int subsetOneMaxValue = totalSum / 2;
        int ans = totalSum + 1;

        for(int j = 0 ; j  <= subsetOneMaxValue; j++){
            if(dp[n][j] == true){
                if(totalSum - 2 * j < ans)  ans = (totalSum - 2 * j);
            }
        }
        return ans;
    }
```

❖ *Count number of Subsets with given difference*

Consider array : [2,4,3,6] Find **number** of subsets possible with given difference **D.**

Let Total Sum of arr[] is → **T,**
We want to find count of subsets possible such that → $\mathbf{S_2 - S_1 = D}$ considering $S_2 > S_1$
Also, $\mathbf{S_2 + S_1 = T}$
Adding,
$\mathbf{2S_2 = T + D}$
⇒ $\mathbf{S_2 = (T + D)/2}$ Problem Reduced to → We want the count of subset with sum S_2 → **TARGET SUM PROBLEM**

```
public static int countPartitions(int n, int d, int[] arr) {

        int MOD = (int) 1e9 + 7;
        int totalSum = 0;
        for (int x : arr) totalSum += x;

        // Check for invalid cases
        if ((totalSum + d) % 2 != 0 || (totalSum + d) / 2 > totalSum)
            return 0;

        int reqSubsetSum = (totalSum + d) / 2;

        // Create 2D DP table
        int[][] dp = new int[n + 1][reqSubsetSum + 1];

        // Base case: Only one way to form sum 0 (empty subset)
        for (int i = 0; i <= n; i++) dp[i][0] = 1;

        // Fill the DP table
        for (int i = 1; i <= n; i++) {
            for (int target = 0; target <= reqSubsetSum; target++) {
                if (arr[i - 1] > target) {
                    dp[i][target] = dp[i - 1][target];
                } else {
                    dp[i][target] = (dp[i - 1][target] + dp[i - 1][target - arr[i - 1]]) % MOD;
                }
            }
        }

        // Return the result from the last cell
        return dp[n][reqSubsetSum];
    }
```

Using 1D dp array

```
public static int countPartitions(int n, int d, int[] arr) {
    int MOD = (int) 1e9 + 7;
    int totalSum = 0;
    for (int x : arr) totalSum += x;

    // Check for invalid cases
    if ((totalSum + d) % 2 != 0 || (totalSum + d) / 2 > totalSum)
        return 0;

    int reqSubsetSum = (totalSum + d) / 2;
    int[] dp = new int[reqSubsetSum + 1];
    dp[0] = 1; // Base case: One way to form subset sum 0 (empty subset)

    // Fill the dp table
    for (int num : arr) {
        for (int target = reqSubsetSum; target >= num; target--) {
            dp[target] = (dp[target] + dp[target - num]) % MOD;
        }
    }

    return dp[reqSubsetSum];
}
```

❖ *Target Sum - [LC 494](#) | Count number of ways to form subset*

Q -> You want to ***build an expression*** out of nums[] by ***adding one of the symbols '+' and '-'*** before each integer in nums and then concatenate all the integers.
Essentially → number of ways to partition subset to form sum k

Same as ⇒ count **#ways** arrays can be divided into 2 subsets such that the difference between them is **D.**
arr = [1,1,2,3] **targetSum = 1;**
eg (+1, -1, -2, +3) == sum = 1
⇒ basically divide elements into 2 subsets (positive numbers and negative numbers) such that the difference is 1.
(1, 3) - (1, 2) = 1

1. Use the **Subset Sum Difference Formula**:

$$S1 - S2 = \text{target}$$

$$S1 + S2 = \text{total}$$

Combine to get:

$$S1 = \frac{\text{total} + \text{target}}{2}$$

This transforms the problem into finding the number of subsets with sum $S1$.

```
class Solution {
    public int findTargetSumWays(int[] nums, int target) {
        int n = nums.length;
        int total = 0;
        for(int x : nums) total += x;

        if(Math.abs(target) > total || (target + total) % 2 != 0) return 0;
        int reqSum = (total + target) / 2;
        int[][] dp = new int[n+1][reqSum + 1];

        // to handle cases if 0 is present in nums.
        dp[0][0] = 1;
        for(int i = 1 ; i <= n ; i++){
            if(nums[i-1] == 0)  dp[i][0] = dp[i-1][0] * 2; // if 0 present #ways doubles (include it or
exclude it)
            else
                dp[i][0] = dp[i-1][0];
        }
        for(int i = 1; i <= n ; i++){
            for(int j = 1 ; j <= reqSum ; j++){
                if(nums[i-1] > j)
                    dp[i][j] = dp[i-1][j];
                else
                    dp[i][j] = dp[i-1][j] + dp[i-1][j - nums[i-1]]; //only difference id ADD all ways
            }
```

```
        }
        return dp[n][reqSum];
    }
}
```

❖ *Coin Change - Minimum #coins to Amount - LC 322 – Good Question*

Initialisation is the key to this question.

Coins_value = [1, 2, 3]
Amount = 5
Question : Given infinite supply of coins, denominations given. Find Minimum number of coins required to form the **amount.**

Initialisation:

	0	1	2	3	4	5
{}	INT_MAX	INT_MAX	INT_MAX	INT_MAX	INT_MAX	INT_MAX
[1]	0					
[1,2]	0					
[1,2,3]	0					

DP[0][1] ⇒ coin[] : [1,2] , amount = 0 → number of coins required → 0.

Dp[0][0] ⇒ coin[] : {} amount =0 . → number of coins required -> INT_MAX
Dp[0][1] ⇒ coin[] : {} amount =1 → number of coins required → **INT_MAX (infinite).**

Min Coins to amount - Recursive

```
class Solution {
    public int coinChange(int[] coins, int amount) {

        Arrays.sort(coins);
        int[][] dp = new int[coins.length+1][amount+1];

        int ans = minCoins(coins.length-1, coins, amount, dp);
        if(ans >= 100000)  return -1;

        return ans;

    }

    int minCoins(int index, int[] coins, int target, int[][] dp){

        if(index == -1)
            return 100000;

        if(target == 0)
            return dp[index][target] = 0;

        if(target % coins[index] == 0)  return dp[index][target] = target/coins[index];

        if(dp[index][target] != 0)
            return dp[index][target];

        int notTake = 0 + minCoins(index-1, coins, target, dp);
        int take = 100000;

        if(target >= coins[index])
            take = 1 + minCoins(index, coins, target- coins[index], dp);

        return dp[index][target] = Math.min(notTake, take);
    }
}
```

Tabulation:

```
class Solution {
    public int getMin(int n) {
        // code here
        int[] coins = new int[]{ 1, 2, 5, 10 };
        int target = n;
        int INF = 1_000_000_000;

        int N = 4; //number of coins
        int[][] dp = new int[N+1][target+1];
        //base case
        dp[0][0] = 0;

        // Base case: 0 coins can't form positive sum
        for (int j = 1; j <= target; j++) dp[0][j] = INF;

        for (int i = 1; i <= N; i++) {
            int coin = coins[i-1];
            for (int j = 1; j <= target; j++) {
                int notTake = dp[i-1][j];
                int take = INF;
                if (j >= coin)
                    take = 1 + dp[i][j - coin];
                dp[i][j] = Math.min(take, notTake);
            }
        }
        return dp[N][target];
    }
}
```

❖ *Number of ways to form sum given coins*

```
import java.util.Arrays;

public class Solution {

    public static long countWaysToMakeChange(int denominations[], int target){
        //write your code here
        int N = denominations.length;
        long[][] dp = new long[N+1][target+1];
        Arrays.sort(denominations);

        //base case : equivalent to in recursion when target==0 return -> 1
        for(int i = 1 ; i <= N; i++)    dp[i][0] = 1;

        for(int i = 1; i <= N; i++){
            int coin = denominations[i-1];
            for(int j = 1 ; j <=target ; j++){
```

```
                long notTake = dp[i-1][j];

                long take = 0;
                if(j >=coin && dp[i][j - coin] != 0)
                    take = dp[i][j-coin];

                dp[i][j] = take + notTake;
            }
        }

        // for(int[] row : dp)
        //  System.out.println(Arrays.toString(row));

        return dp[N][target];

    }

}
```

❖ Parent :Unbounded Knapsack Problem :

For unbounded knapsack

– where multiple times you can take an element → perform forward operation

```
class Solution {
    static int knapsack(int W, int val[], int wt[]) {
        int N = val.length;
        int[] dp = new int[W + 1];  // dp[capacity] = max value for given capacity

        for (int i = 0; i < N; i++) {
            int weight = wt[i];
            int value = val[i];

            // iterate forward so that we take multiple instances of element → unbounded knapsack
            for (int capacity = weight; capacity <= W; capacity++) {
              dp[capacity] = Math.max(dp[capacity], value + dp[capacity - weight]);
            }
        }
        return dp[W];
    }
}
```

Unbounded Knapsack (forward loop) →

- capacity=2 → dp[2]=2
- capacity=4 → dp[4]=4 (taking item twice) → 2 + dp[4 - 2]
- capacity=6 → dp[6]=6
- capacity=8 → dp[8]=8

Recursive:

```
class Solution {
    static int knapSack(int val[], int wt[], int capacity) {
        // code here
        int N = val.length;
        int[][] dp = new int[N+1][capacity+1];
        for(int[] row : dp) Arrays.fill(row, -1);
        return addItem(N-1, val, wt, capacity, dp);
    }

    private static int addItem(int idx, int[] value, int[] weight, int capacity, int[][] dp){

        if(capacity == 0)
            return 0;

        if(idx < 0)
            return 0;

        if(dp[idx][capacity] != -1)
            return dp[idx][capacity];

        int notTake = addItem(idx - 1, value, weight, capacity, dp);

        int take = 0;
        if(weight[idx] <= capacity)
            take = value[idx] + addItem(idx, value, weight, capacity - weight[idx], dp);

        return dp[idx][capacity] = Math.max(take, notTake);
    }
}
```

1. Rod Cutting problem :

Given a rod length N, each length is associated with some value,
You need to divide the rod in such a way that the sum of length of all small parts = N and value obtained is maximum.
Same as unbounded knapsack

Recursive:

```
class Solution {
    public int cutRod(int[] price) {
        // code here
        //unbounded knapsack
        int N = price.length;
        return getMaxValue(N-1, price, N);
    }

    private int getMaxValue(int idx, int[] price, int capacity){

        if(idx < 0)
            return 0;
        if(capacity == 0)
            return 0;

        int notTake = getMaxValue(idx - 1, price, capacity);
        int take = 0;
        if((idx + 1) <= capacity)
            take = price[idx] + getMaxValue(idx, price, capacity - (idx + 1));

        return Math.max(notTake, take);
```

```
    }
}
```

Memorisation

```
class Solution {
    public int cutRod(int[] price) {
        // code here
        //unbounded knapsack
        int N = price.length;
        int[][] dp = new int[N][N+1];
        for(int[] row: dp)  Arrays.fill(row, -1);

        return getMaxValue(N-1, price, N, dp);
    }

    private int getMaxValue(int idx, int[] price, int capacity, int[][] dp){

        if(idx < 0)
            return 0;
        if(capacity == 0)
            return 0;
        if(dp[idx][capacity] != -1)
            return dp[idx][capacity];

        int notTake = getMaxValue(idx - 1, price, capacity, dp);
        int take = 0;
        if((idx + 1) <= capacity)
            take = price[idx] + getMaxValue(idx, price, capacity - (idx + 1), dp);

        return dp[idx][capacity] = Math.max(notTake, take);
    }
}
```

Tabulation:

```
class Solution {
    public int cutRod(int[] price) {
        // code here
        //unbounded knapsack
        int N = price.length;
        int[][] dp = new int[N+1][N+1];

        for(int idx = 1 ; idx <= N ; idx++){
            for(int len = 1 ; len <= N ; len++){

                int notTake = dp[idx - 1][len];
                int take = 0;
                if( idx <= len)
                    take = price[idx-1] + dp[idx][len - idx];

                dp[idx][len] = Math.max(notTake, take);
            }
        }

        return dp[N][N];
    }

}
```

Space Optimised to only one 1D array. (since we are using only one guy (top) from prev arr[])

```
class Solution {
    public int cutRod(int[] price) {
        // code here
        //unbounded knapsack
        //space optimised
        int N = price.length;
        int[] prev = new int[N+1];

        for(int idx = 1 ; idx <= N ; idx++){
            for(int len = 1 ; len <= N ; len++){

                int notTake = prev[len];
                int take = 0;
                if( idx <= len)
                    take = price[idx-1] + prev[len - idx];

                prev[len] = Math.max(notTake, take);
            }
        }

        return prev[N];
    }

}
```

❖ Parent : Longest Common Subsequence: DP ON STRINGS

Type of Question:
Comparison
Replace / Edit in string.

Types of Question:

1. Longest Common Subsequence(LCS)
2. Print LCS
3. Shortest Common super-sequence (SCS)
4. Print SCS
5. Minimum number of insertion/deletion to make string a → b
6. Largest repeating subsequence
7. Length of longest subsequence if a which is a substring in b
8. Subsequence Pattern Matching
9. Count how many times a appears as subsequence in b
10. Largest **palindromic** subsequence
11. Largest **Palindromic** substring
12. Count of **Palindromic** Substring
13. Minimum number of deletions in substring to make it **palindrome**
14. Minimum number of insertions in substring to make it **palindrome**

Difference b/w substring and subsequence:

- **Substring** are **continuous** part of a string
- **Subsequences** *need not be* ***continuous*** but relative order of elements are the same.

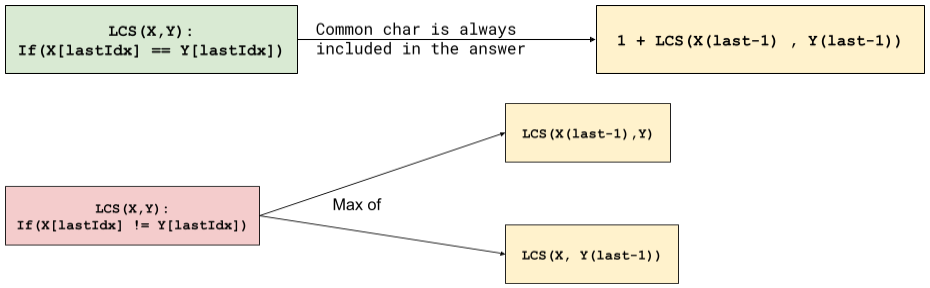
Rule:

- Keep 2 index { idx1, idx2 } (one for each string to compare)
- Explore all possibilities

❖ *Longest Common Subsequence :*

Given 2 strings, find the longest common subsequence.

X : a b c d g h
Y : a b e d f h a
→ LCS (X, Y)→ a b d h



X: "abef" → length: 4
Y: "abcdaf" → length= 6
We create a dp arr of length 5 X 7 as, (4+1) and (6+1)

If
X.charAt(i) == Y.charAt(j)
then include that in ans → 1 + LCS[i-1][j-1]
Else
Then LCS → Max (excluding Y → LCS[i][j-1] , excluding X → LCS[i-1][j])

X ↓ / Y →	0 (null)	1(a)	2(b)	3(c)	4(d)	5(a)	6(f)
0 (null)	When X = null and Y== null → LCS = 0	0	0	0	0	0	0
1(a)	When X len = 1 and Y== null → LCS = 0	1	1	1	1	1	1
2(b)	0	Max(dp[i-1][j], dp[i][j-1]) = 1	1 + dp[i-1][j-1] = 2	2	2	2	2
3(e)	0	1	2	2	2	2	2
4(f)	0	1	2	2	2	2	3

Recursive solution:

```
class LCS {
    public int lenCommonSubstring(String X, String Y, int n, int m){
        if(n == 0 || m == 0)
            return 0;

        if(X.charAt(n-1) == Y.charAt(m-1))
            return 1 + lenCommonSubstring(X, Y, n-1, m-1);
        else
            return Math.max(lenCommonSubstring(X, Y, n, m-1), lenCommonSubstring(X, Y, n-1, m));
    }
}
```

Memorised Solution Top-down:

```
class Solution {
    static int lcs(String s1, String s2) {
        // code here
        int N = s1.length();
        int M = s2.length();

        int[][] dp = new int[N][M];
        for(int[] row : dp) Arrays.fill(row, -1);

        return processLCS(s1, s2, N-1, M-1, dp);
    }

    private static int processLCS(String s1, String s2, int idx1, int idx2, int[][] dp){

        if(idx1 < 0 || idx2 < 0)
            return 0;

        if(dp[idx1][idx2] != -1)
            return dp[idx1][idx2];

        if(s1.charAt(idx1) == s2.charAt(idx2))
            return dp[idx1][idx2] =  1  + processLCS(s1, s2, idx1 - 1, idx2 - 1, dp);

        return dp[idx1][idx2] = Math.max( processLCS(s1, s2, idx1-1, idx2, dp),
                                          processLCS(s1, s2, idx1, idx2-1, dp));
    }
}
```

Tabulation- Bottom-up:

```
class Solution {
    public int longestCommonSubsequence(String text1, String text2) {
        int n = text1.length();
        int m = text2.length();

        int dp[][] = new int[n+1][m+1]; //by default all elements are 0 in int[]

        for(int i = 1 ; i < n+1 ; i++){
            for(int j = 1 ; j < m+1 ; j++){
                if(text1.charAt(i-1) == text2.charAt(j-1))
                    dp[i][j] = 1 + dp[i-1][j-1];
                else
                    dp[i][j] = Math.max(dp[i][j-1], dp[i-1][j]);
            }
        }
        return dp[n][m];

    }
}
```

Space Optimised

```
class Solution {
    static int lcs(String s1, String s2) {
        // code here
        int N = s1.length();
        int M = s2.length();
        int[] prev = new int[M+1];

        for(int i = 1; i <= N ; i++){
            int[] curr = new int[M+1];
            for(int j = 1; j <= M ; j++){
                if(s1.charAt(i-1) == s2.charAt(j-1))
                    curr[j] = 1 + prev[j-1];

                else
                    curr[j] = Math.max(prev[j], curr[j-1]);
```

```
            }

            prev=curr;
        }

        return prev[M];
    }
}
```

❖ *Print LCS*

```
import java.util.Arrays;
import java.lang.StringBuilder;

public class Solution {
    public static String findLCS(int n, int m, String s1, String s2){
        // Write your code here.
        int N = s1.length();
        int M = s2.length();

        int[][] dp = new int[N+1][M+1];

        for(int i = 1; i <=N ; i++)
        {
            for(int j = 1; j <= M; j++)
            {
                if(s1.charAt(i-1) == s2.charAt(j-1))
                    dp[i][j] = 1 + dp[i-1][j-1];

                else
                    dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
            }
        }

        // for(int[] row : dp)
        //     System.out.println(Arrays.toString(row));

        if(dp[N][M] == 0)
            return "";

        //BUILDING LCS
        StringBuilder sb = new StringBuilder();

        int x = N, y = M;
        while(x != 0 && y != 0){
            if(dp[x][y] > dp[x-1][y] && dp[x][y] > dp[x][y-1]){/or→ s1.charAt(x-1)==s2.charAt(y-1)
                                                                //both works
                sb.append(s1.charAt(x-1));
                x--;
                y--;
            } else if(dp[x-1][y] > dp[x][y-1]){ //check from where it is coming, from left or top
                x--;
            } else {
                y--;
            }
```

```
        }

        sb.reverse();
        return sb.toString();

    }
}
```

❖ *Longest Common Substring:*

Q. How did we identify that it is a LCS derived problem ? → Input format is same, processing is almost same and also o/p format is same.

In matching substring whenever there is a match
- Update matching string

If not match
- Reset the currentMatch streak.

Recursive:

```
public static int lcs(String str1, String str2){
        // Write your code here. ==> SUBSTRING : discontinuous not allowed

        int n = str1.length();
        int m = str2.length();

        return findSol(str1, str2, n, m, 0);
    }

    static int findSol(String X, String Y, int n, int m, int matchingStreak){

        //Base case:
        if(n==0 || m==0)    return matchingStreak;

        int currStreak = matchingStreak;

        if(X.charAt(n-1) == Y.charAt(m-1))
            currStreak = findSol(X, Y, n-1, m-1, matchingStreak + 1);

        //in every case compare all cases.
        // branching resets streak
        int left = findSol(X, Y, n - 1, m, 0);
        int right = findSol(X, Y, n, m - 1, 0);

        return Math.max(currStreak, Math.max(left, right));

    }
```

Memorisation:

```
static int[][][] dp;

static int findSol(String X, String Y, int n, int m, int streak){
    if(n == 0 || m == 0) return streak;
    if(dp[n][m][streak] != -1) return dp[n][m][streak];

    int currStreak = streak;
    if(X.charAt(n-1) == Y.charAt(m-1))
```

```
        currStreak = findSol(X, Y, n-1, m-1, streak+1);

    int left = findSol(X, Y, n-1, m, 0);
    int right = findSol(X, Y, n, m-1, 0);

    return dp[n][m][streak] = Math.max(currStreak, Math.max(left, right));
}
```

Tabulation:
//BEST HERE → no use of 3D dp

```
public class Solution {
    public static int longestCommonSubstring(String str1, String str2) {
        int n = str1.length();
        int m = str2.length();

        int[][] dp = new int[n+1][m+1]; // +1 to handle base cases easily
        int maxLen = 0;

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= m; j++) { //considering str2(1→j) what is the match with str1(1→i)
                if (str1.charAt(i-1) == str2.charAt(j-1)) {
                    dp[i][j] = 1 + dp[i-1][j-1];
                    maxLen = Math.max(maxLen, dp[i][j]);
                } else {
                    dp[i][j] = 0; // reset streak if doesn't match.
                }
            }
        }

        return maxLen;
    }
}
```

❖ *Longest Palindromic Subsequence (LPS)*

Steps:
- Reverse the String
- LCS (str, reverse(str)) → LPS

Note: LCS (str, reverse(str)) → ***ALWAYS*** form a palindromic string.

```
class Solution {
    public int longestPalindromeSubseq(String s) {
        int N = s.length();
        String sRev = new StringBuilder(s).reverse().toString();

        int[][] dp = new int[N+1][N+1];

        for(int i = 1 ; i <= N; i++){
            for(int j = 1; j <= N ; j++){

                if(s.charAt(i-1) == sRev.charAt(j-1))
                    dp[i][j] = 1 + dp[i-1][j-1];
                else
                    dp[i][j] = Math.max(dp[i][j-1], dp[i-1][j]);
            }
        }
        return dp[N][N];
    }
}
```

❖ *Longest Palindromic SubString*

Not a DP problem but very interesting
All single chars are palindrome
Expand neighbours until we see palindrome condition is met successfully
→ check even length
→ odd length

```
class Solution {
    public String longestPalindrome(String s) {

        int start = 0;
        int end = 0;

        for (int i = 0; i < s.length(); i++) {
            int len1 = expand_neighbours(s, i, i);     // odd-length
            int len2 = expand_neighbours(s, i, i + 1); // even-length

            int len = Math.max(len1, len2);

            if (len > end - start + 1) {
                start = i - (len - 1) / 2;
                end = i + len / 2;
            }
        }

        return s.substring(start, end + 1);
    }

    int expand_neighbours(String s, int left, int right) {
        while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
            left--;
            right++;
        }
        return right - left - 1; // corrected length
    }
}
```

❖ *Minimum insertion to make a String Palindrome*

Consider String s = "abcaa"
LCS -> a b a or a a a
What we need to do we need to add missing chars at some position to make full string palindrome
→ a b c a b c a
Here b c added which is nothing but left out chars from LCS.

Count (#chars needs to be added to make string Palindrome) → N - len(LCS)

```
class Solution {
    public int minInsertions(String s) {
        String sRev = new StringBuilder(s).reverse().toString();

        if(s.equals(sRev))  return 0; //already a palindrome
```

```java
        int N = s.length();

        //LCS
        int[][] dp = new int[N+1][N+1];

        for(int i = 1 ; i <= N ; i++){
            for(int j = 1; j <= N; j++){

                if(s.charAt(i-1) == sRev.charAt(j-1))
                    dp[i][j] = 1 + dp[i-1][j-1];
                else
                    dp[i][j] = Math.max(dp[i][j-1], dp[i-1][j]);
            }
        }

        int longestPalindromicSubsequence = dp[N][N];
        int result = N - longestPalindromicSubsequence;

        return result;

    }
}
```

❖ ***Minimum insertions/deletions to make strings equal***

```java
class Solution {
    public int minDistance(String word1, String word2) {
        int N = word1.length();
        int M = word2.length();

        //LCS
        int[][] dp= new int[N+1][M+1];
        for(int i = 1; i <= N; i++){
            for(int j = 1; j <= M; j++){
                if(word1.charAt(i-1) == word2.charAt(j-1))
                    dp[i][j] = 1 + dp[i-1][j-1];
                else
                    dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
            }
        }

        int lcsLength = dp[N][M];
        int uncommonChars = N + M - 2 * lcsLength;

        return uncommonChars;
    }
}
```

❖ ***Shortest Common SuperSequence*** *(SCS)*

superSequence -> String whose subsequence contains both str1 and str2

Eg. str1 = "abac" and str2 = "cab"

Then res = "cabac" → is once supersequence

Idea →

- Get the lcsString
- Add the non-lcs chars while building an LCS string.

```java
class Solution {
    public String shortestCommonSupersequence(String str1, String str2) {
        //superSequence -> String whose subsequence contains both str1 and str2

        int N = str1.length();
        int M = str2.length();
        //LCS
        int[][] dp = new int[N+1][M+1];
        for(int i = 1 ; i <=N ; i++){
            for(int j = 1; j <= M; j++){
                if(str1.charAt(i-1) == str2.charAt(j-1))
                    dp[i][j] = 1 + dp[i-1][j-1];
                else
                    dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
            }
        }

        //form the SCS string;
        StringBuilder sb = new StringBuilder();
        int i = N, j = M;
        while(i >= 1 && j >= 1){
            if(str1.charAt(i-1) == str2.charAt(j-1)){
                sb.append(str1.charAt(i-1));  //LCS chars
                i--;
                j--;
            } else if(dp[i-1][j] > dp[i][j-1]){ //moving to top row, so add the current 'i'
                sb.append(str1.charAt(i-1));  //non-lcs chars
                i--;
            } else{ //moving to left column, so add the current 'j'
                sb.append(str2.charAt(j-1)); //non-lcs chars
                j--;
            }
        }
 // if some non-lcs chars remaining add that also
        while(i >= 1){
            sb.append(str1.charAt(i-1));
            i--;
        }
        while(j >= 1){
            sb.append(str2.charAt(j-1));
            j--;
        }

        return sb.reverse().toString();
    }
}
```

❖ Distinct Subsequence – how many times str2 appears in str1

Eg , str1 = "babgbag" str2 = "bag" → result #count = 3

Here we want ALL occurrence

Idea is

From str(1 → i) check if last char match with last char of str2(1 → j);

Count Ways Question : → General Solution Struncture

F()

```
{
    Base Case
        Return 1 or 0

    Return F() + F() + …. + F() //all possible ways
}
```

- Intuition

At each step, when characters

match (`s[i] == t[j]`), you have **two choices**:

1. Use this character to match (`take`)
2. Skip it (`not take`)

If they don't match

you can only skip from `s`.

Memorisation:

```
class Solution {
    public int numDistinct(String s, String t) {
        //we basically check here how many times we can make distinct subsequences from String S to form String t.
        int N = s.length();
        int M = t.length();

        int[][] dp = new int[N+1][M+1];
        for(int[] row : dp) Arrays.fill(row, -1);

        return countSubSeq(s, t, N, M, dp);
    }

    private int countSubSeq(String s, String t, int i, int j, int[][] dp){
        //considering 1 based indexing for strings.
        if(j == 0 )
            return 1; //we matched all chars from t in subsequence

        if(i == 0 )
            return 0; //before all match from t, str s is exhausted.

        if(dp[i][j] != -1)
            return dp[i][j];

        if(s.charAt(i-1) == t.charAt(j-1))
        {
            int take = countSubSeq(s, t, i-1, j-1, dp);
            int notTake = countSubSeq(s, t, i-1, j, dp);

            return dp[i][j] =  take  + notTake;
        } else
            return dp[i][j] = countSubSeq(s, t, i-1, j, dp);   //not take

    }
}
```

`Say if s = "bab", t = "ab"`

- We want **ways to form "ab" from "bab"**.

`dp[i][j]` = number of ways to form `t[0..j-1]` **from** `s[0..i-1]`.

```
class Solution {
    public int numDistinct(String s, String t) {
        //we basically check here how many times we can make distinct subsequences from String S to form String t.
        int N = s.length();
        int M = t.length();

        int[][] dp = new int[N+1][M+1];

        for(int i = 0 ; i <= N ; i++) dp[i][0]  = 1; // if t="" we can form it from any s (by removing all).
        for(int j = 1 ; j <= M ; j++) dp[0][j] = 0; //given s="" we cannot form t(non empty).

        for(int i=1 ; i  <= N ; i++){
            for(int j=1 ; j <= M ; j++){

                if(s.charAt(i-1) == t.charAt(j-1))
                    dp[i][j] = dp[i-1][j-1]  + dp[i-1][j];
                else
                    dp[i][j] = dp[i-1][j];
            }
        }
        return dp[N][M];
    }
}
```

2. *Minimum Operations to make Word1 → Word2* | *Edit Distance LC-72*

Allowed 3 operations :

I. Insert a char → when we insert the inserted char is matched to the char in word2 under consideration. Thus i remains same as we inserted a new char at that position and j → j-1
II. Delete a char → we ignore the char in word1 and move to 1 index left. i.e (i-1)
III. Replace a char → we replace with matching char from word2 and move (i-1) and (j-1).

Memorised:

```
class Solution {
    public int minDistance(String word1, String word2) {
        int N = word1.length();
        int M = word2.length();

        int[][] dp = new int[N+1][M+1];
        for(int[] row : dp) Arrays.fill(row, -1);

        return getMin(word1, word2, N,M, dp);
    }

    int getMin(String word1, String word2, int i, int j, int[][] dp){
        if(i == 0)
            return j;
        if(j == 0)
            return i;

        if(dp[i][j] != -1)
            return dp[i][j];

        if(word1.charAt(i-1) == word2.charAt(j-1))
            return dp[i][j] = getMin(word1, word2, i-1, j-1, dp);

        int insert = 1 + getMin(word1, word2, i, j-1, dp);  //inserted chat in word1 which will be a match to char at word2(j-1)
        int delete = 1 + getMin(word1, word2, i-1, j, dp); //delete from word1 means move i -> i-1
        int replace = 1 + getMin(word1, word2, i-1, j-1, dp); //replace will make word1(i) == word2(j)

        return dp[i][j] = Math.min(insert, Math.min(delete, replace));
```

```
    }
}
```

Tabulation:

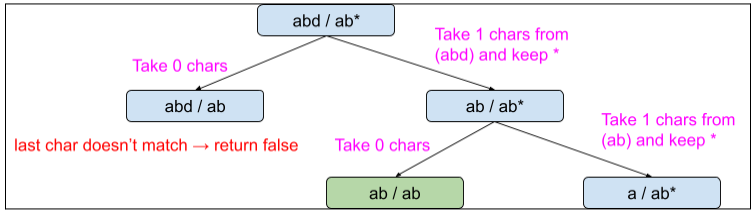
Here check how base case we derive from recursion code itself

```
class Solution {
    public int minDistance(String word1, String word2) {
        int N = word1.length();
        int M = word2.length();

        int[][] dp = new int[N+1][M+1];

        for(int j = 0; j <= M ; j++)  dp[0][j] = j; //equivalent to -> if(i == 0) return j; (in recursion)
        for(int i = 0 ; i <= N ; i++) dp[i][0] = i; //equivalent to -> if(j == 0) return i; (in recursion)

        for(int i = 1; i <= N; i++ ){
            for(int j = 1; j <= M ; j++){
                if(word1.charAt(i-1) == word2.charAt(j-1))
                    dp[i][j] = dp[i-1][j-1];
                else {
                    int insert = 1 + dp[i][j-1];
                    int delete = 1 + dp[i-1][j];
                    int replace = 1 + dp[i-1][j-1];

                    dp[i][j] =  Math.min(insert, Math.min(delete, replace));
                }
            }
        }

        return dp[N][M];
    }

}
```

3. *Wild Card Matching*

Two strings given S and P.
S -> source String (abd)
P -> pattern string (ab*)

We **check from last chars** from each string
If match → reduce by 1 for both
Else
—> check further conditions (check in code)



Eventually **ab/ab** will match

Memorisation

```
class Solution {
    public boolean isMatch(String s, String p) {
        int N = s.length();
        int M = p.length();
        Boolean[][] dp = new Boolean[N+1][M+1];

        return checkMatch(s, p, N, M, dp);
    }

    private boolean isAllStarsRemaining(String p, int end){
        //check if pattern only contains stars --> "****"
        for(int idx = 0 ; idx < end; idx++)
            if(p.charAt(idx) != '*') return false;

        return true;
    }

    private boolean checkMatch(String s, String p, int i, int j, Boolean[][] dp){

        //base case i==0 || j==0
        if(i == 0 && j == 0)
            return true;
        if(j == 0) //p->exhausted
            return false;

        if(dp[i][j] != null)
            return dp[i][j];

        if(i == 0 && j != 0)  // S -> exhausted by P remaining
            return dp[i][j] = isAllStarsRemaining(p, j); //checking case: S -> "" and P -> "***.." (one or many)

        if(s.charAt(i - 1) == p.charAt(j-1) || p.charAt(j-1) == '?')
           return dp[i][j] =  checkMatch(s, p, i-1, j-1, dp);

        else if(p.charAt(j-1) == '*'){
            // option-1 : take 0 chars
            boolean option1 = checkMatch(s, p, i, j-1, dp);

            //option-2 : keep '*' and delete 1 char from s
            boolean option2 = checkMatch(s, p, i-1, j, dp);

            return dp[i][j] = (option1 || option2);
        }
        else
            return dp[i][j] = false; //s.charAt(i-1) != p.charAt(j-1)

    }
}
```

Tabulation:

```
class Solution {
    public boolean isMatch(String s, String p) {
        int N = s.length();
        int M = p.length();
        boolean[][] dp = new boolean[N+1][M+1];

        dp[0][0] = true;
        for(int i = 1; i <=N ; i++) dp[i][0] = false;
        for(int j = 1; j <= M; j++) dp[0][j] = isAllStarsRemaining(p, j);

        for(int i = 1 ; i <= N; i++){
```

```
            for(int j=1; j <= M; j++){
                if(s.charAt(i - 1) == p.charAt(j-1) || p.charAt(j-1) == '?')
                    dp[i][j] =  dp[i-1][j-1];

                else if(p.charAt(j-1) == '*')
                    dp[i][j] = dp[i][j-1] ||  dp[i-1][j];

                else
                    dp[i][j] = false;

            }
        }
        return dp[N][M];
    }

    private boolean isAllStarsRemaining(String p, int end){
        //check if pattern only contains stars --> "****"
        for(int idx = 0 ; idx < end; idx++)
            if(p.charAt(idx) != '*') return false;

        return true;
    }
}
```

Dynamic Programming - 3 (DP on STOCKS)

1. ***Best time to buy and sell stocks only once.***

Idea is :

If you want to sell stock at j where j > i

→ you want to see what is the minimum purchase cost in left.

```
class Solution {
    public int maxProfit(int[] prices) {
        int profit = 0;
        int minValueLeft = prices[0];

        for(int i = 1; i < prices.length; i++){
            int currProfit = prices[i] - minValueLeft;
            profit = Math.max(profit, currProfit);
            minValueLeft = Math.min(minValueLeft, prices[i]);
        }

        return profit;
    }
}
```

2. ***Best time to buy and sell stocks - 2***

Variation 2 : You can buy and sell stocks multiple times. (you need to buy first to sell)

Eg : prices : [7,1,5,3,6,4]
Here if you buy on day 2 and sell on day 3 → profit = +3 - 1

This question becomes forming a max sum where consecutive elements are of opposite sign.

{ -7, +1, -5, +3...}

→ best ans → {-1, +5, -3, +6}

```
class Solution {
    public int maxProfit(int[] prices) {
        int N = prices.length;
        int[][] dp = new int[N][2];
        for(int[] row : dp) Arrays.fill(row, -1);

        return Math.max(0, process(prices, 0, 0, dp));
    }

    //turn -> {0: buy, 1: sell}
    private int process(int[] prices, int idx, int turn, int[][] dp){
        if(idx == prices.length)
            return 0;

        if(dp[idx][turn] != -1)
            return dp[idx][turn];

        int take = 0;
        if(turn == 0){
            //buy
            take = -prices[idx] + process(prices, idx + 1, 1, dp);
        } else {
            //sell
            take = prices[idx] + process(prices, idx + 1, 0, dp);
        }

        int notTake = process(prices, idx + 1, turn, dp);

        return dp[idx][turn] = Math.max(take, notTake);
    }
}
```

Best solution:

Start from end(N) to process sell first and go to (0)

```
class Solution {
    public int maxProfit(int[] prices) {
        int maxProfit = 0;
        for(int i = prices.length -1 ; i > 0 ; i--){
            if(prices[i] > prices[i - 1])
                maxProfit += (prices[i] - prices[i - 1]);

        }
        return maxProfit;
    }
}
```

3. ***Best Time to buy and sell stock in only 2 complete transactions – 3D dp***

Memorisation

```
class Solution {
    public int maxProfit(int[] prices) {
        int N = prices.length;
        int maxTrxn = 4;// (2 buy, 2 sell)
        int[][][] dp = new int[N+1][2][maxTrxn+1];
        for(int[][] matrix : dp){
            for(int[] row : matrix)
                Arrays.fill(row, -1);
        }
        return process(prices, 0, 0, maxTrxn, dp);
    }
```

```java
    private int process(int[] prices, int idx, int type, int currTrxn, int[][][] dp){
        if(idx == prices.length)
            return 0;

        if(currTrxn == 0)
            return 0;

        if(dp[idx][type][currTrxn] != -1)
            return dp[idx][type][currTrxn];

        int take = 0;
        if(type == 0){
            //buy
            take = -prices[idx] + process(prices, idx + 1, 1, currTrxn - 1, dp);
        } else {
            //type-> 1 sell
            take = prices[idx] + process(prices, idx + 1, 0, currTrxn - 1, dp);
        }

        int notTake = process(prices, idx + 1, type, currTrxn, dp);

        return dp[idx][type][currTrxn] =  Math.max(take, notTake);
    }
}
```

Tabulation

```java
class Solution {
    public int maxProfit(int[] prices) {
        int N = prices.length;
        int maxTrxn = 4;// (2 buy, 2 sell)
        int[][][] dp = new int[N+1][2][maxTrxn+1];

        //BASE case from recursion; NOT required since already initialise to 0
        // but see how its written from recursion

        // if(currTrxn == 0)   return 0;
        for(int i = 0 ; i <=N ; i++){
            for(int buy = 0; buy < 2 ; buy++)
                dp[i][buy][0] = 0;
        }
        // if(idx == prices.length) return 0;
        for(int buy = 0 ; buy < 2; buy++)
            for(int trxn = 0; trxn <= maxTrxn; trxn++)
                dp[N][buy][trxn] = 0;

        // Fill table backwards
        for(int i = N - 1; i >= 0; i--){
            for(int buy = 0 ; buy < 2; buy++){
                for(int trxn = 1 ; trxn <= maxTrxn ; trxn++){
                    int take;
                    if(buy == 0){
                        // BUY action
                        take = -prices[i] + dp[i + 1][1][trxn - 1];
                    } else {
                        // SELL action
                        take = prices[i] + dp[i + 1][0][trxn - 1];
                    }

                    int notTake = dp[i + 1][buy][trxn];

                    dp[i][buy][trxn] =  Math.max(take, notTake);
                }
            }
        }
        return dp[0][0][maxTrxn];
    }
}
```

Step-by-step for `[4, 7]`

Base row: `dp[2][*][*] = 0` (past the last day).

i = 1 (price = 7):

- `buy = 0` (can buy):
  `take = -7 + dp[2][1][t-1] = -7 + 0 = -7`
  `notTake = dp[2][0][t] = 0`
  ⇒ `dp[1][0][t] = 0` (buying on the last day is bad)

- `buy = 1` (can sell):
  `take = +7 + dp[2][0][t-1] = 7 + 0 = 7`
  `notTake = dp[2][1][t] = 0`
  ⇒ `dp[1][1][t] = 7` (if you already hold, sell now)

i = 0 (price = 4):

- `buy = 0` (can buy):
  `take = -4 + dp[1][1][t-1] = -4 + 7 = 3`
  `notTake = dp[1][0][t] = 0`
  ⇒ `dp[0][0][t] = 3`

- `buy = 1` (can sell):
  `take = +4 + dp[1][0][t-1] = 4 + 0 = 4`
  `notTake = dp[1][1][t] = 7`
  ⇒ `dp[0][1][t] = 7`

Answer: `dp[0][0][4] = 3`, i.e., **buy at 4, sell at 7.**

4. *Buy and sell stocks IV - (there must be gap of 1 day b/w two consecutive sell and buy)*

Memorisation

```java
class Solution {
    public int maxProfit(int[] prices) {
        int N = prices.length;
        //0->sell, 1-> buy
        int[][] dp = new int[N+1][2];
        for(int[] row : dp) Arrays.fill(row, -1);

        return process(prices, 0, 1, dp);
    }

    int process(int[] prices, int idx, int buy, int[][] dp){
        if(idx >= prices.length)
            return 0;

        if(dp[idx][buy] != -1)
            return dp[idx][buy];

        if(buy == 1)
        {
            //BUY trxn
            return dp[idx][buy] = Math.max( -prices[idx] + process(prices, idx + 1, 0, dp),
                                0 + process(prices, idx + 1, 1, dp));
```

```
        } else {
            //Sell trxn
            return dp[idx][buy] = Math.max( prices[idx] + process(prices, idx + 2, 1, dp), //buy after 1 day
                            0 + process(prices, idx + 1, 0, dp)) ;
        }


    }
}
```

Tabulation:

```
class Solution {
    public int maxProfit(int[] prices) {
        int N = prices.length;
        //0->sell, 1-> buy
        int[][] dp = new int[N+2][2];

        for(int i = N-1; i >= 0 ; i--){
            for(int buy = 0 ; buy < 2 ; buy++){

                if(buy == 1)
                {
                    //BUY trxn
                    dp[i][buy] = Math.max(-prices[i] + dp[i+1][0], dp[i + 1][1]);
                } else {
                    //Sell trxn
                    dp[i][buy] = Math.max( prices[i] + dp[i + 2][1], dp[i + 1][0]) ;
                }

            }
        }
        return dp[0][1];
    }

}
```

5. [Buy and sell stocks 5 - (every sell have some fee)]

```
class Solution {
    public int maxProfit(int[] prices, int fee) {
        int N = prices.length;
        int[][] dp = new int[N+1][2];
        for(int[] row : dp) Arrays.fill(row, -1);

        return process(prices, fee, 0, 1, dp); //buy:1 , sell:0
    }

    int process(int[] prices, int fee, int idx, int buy, int[][] dp){
        if(idx == prices.length)
            return 0;

        if(dp[idx][buy] != -1)
            return dp[idx][buy];

        if(buy == 1){
            return dp[idx][buy] =  Math.max( -prices[idx] + process(prices, fee, idx + 1, 0, dp),
                            0 + process(prices, fee, idx + 1, 1, dp));
        } else {
            return dp[idx][buy] = Math.max( prices[idx] - fee + process(prices, fee, idx + 1, 1, dp) ,
                                    0 + process(prices, fee, idx + 1, 0, dp));
        }
    }
}
```

Dynamic Programming - 4 (Longest Increasing Subsequence)

Longest increasing subsequence

```
class Solution {
    public int lengthOfLIS(int[] nums) {
        int N = nums.length;
        int[][] dp = new int[N+1][N+1];
        for(int[] row : dp) Arrays.fill(row, -1);

        //1-base indexing
        return LIS(nums, 1, 0, dp);
    }

    int LIS(int[] nums, int curr, int prev, int[][] dp){
        // System.out.println(curr + " : " + prev);
        if(curr > nums.length)
            return 0;

        if(dp[curr][prev] != -1)
            return dp[curr][prev];

        //strictly increasing
        int take = (prev == 0 || nums[curr - 1] > nums[prev - 1]) ? 1 + LIS(nums, curr + 1, curr ,dp) : 0;
        int notTake = LIS(nums, curr + 1, prev, dp);

        return dp[curr][prev] = Math.max(take, notTake);
    }
}
```

Tabulation:

```
class Solution {
    public int lengthOfLIS(int[] nums) {
        int N = nums.length;
        int[][] dp = new int[N+2][N+2];

//selection of curr depends on prev → hence starting from backwards
        for (int curr = N; curr >= 1; curr--) {
            for (int prev = curr - 1; prev >= 0; prev--) {

                // Option 1: Take current element if it's greater than last picked
                int take = 0;
                if (prev == 0 || nums[curr - 1] > nums[prev - 1]) {
                    take = 1 + dp[curr + 1][curr];
                }

                // Option 2: Skip current element
                int notTake = dp[curr + 1][prev];

                // Choose the best
                dp[curr][prev] = Math.max(take, notTake);

            }
        }

        return dp[1][0]; // starting with curr=1 and prev=0
    }
}
```

Tabulation : Special unrelated approach
Dp[i] → length of Longest increasing subsequence ending at index i.

```
class Solution {
```

```java
    public int lengthOfLIS(int[] nums) {
        int N = nums.length;
        int[] dp = new int[N];
        Arrays.fill(dp, 1);     // all nums[i] can end with itself.

        int maxi = 1;
        for(int i = 0 ; i < N ; i++){
            for(int prev = 0; prev < i; prev++){
                if(nums[prev]  < nums[i] && dp[i] < 1 + dp[prev]){
                    dp[i] = 1 + dp[prev];
                }
            }
            maxi = Math.max(maxi, dp[i]);
        }

        return maxi;
    }
}
```

Longest Divisible Subset:

Q. Given nums[] = { 1, 16, 4, 7, 8}

Given a set of distinct positive integers nums, return the largest subset answer such that every pair (answer[i], answer[j]) of elements in this subset satisfies:

answer[i] % answer[j] == 0, or

answer[j] % answer[i] == 0

Soln:

After sorting -> {1, 4, 7, 8, 16} ⇒ question becomes similar to LIS. only the checking condition gets changed.

If `nums[curr] % nums[prev] == 0` → add in dp matrix

```java
class Solution {
    public List<Integer> largestDivisibleSubset(int[] nums) {

        Arrays.sort(nums);
        int n  = nums.length;
        int[] dp = new int[n];
        int[] hash = new int[n];

        int maxValue = 0;
        int lastIdx = 0;

        for(int i = 0; i < n; i++){
            hash[i] = i;
            for(int prev = 0; prev < i; prev++){
                if(nums[i] % nums[prev] == 0 && dp[i] < dp[prev] + 1){
                    dp[i] = 1 + dp[prev];
                    hash[i] = prev;
                }
            }

            if(dp[i] > maxValue){
                maxValue = dp[i];
                lastIdx = i;
            }
        }

// Printing the result.
        List<Integer> result = new ArrayList<>();
        while(hash[lastIdx] != lastIdx){
            result.add(nums[lastIdx]);
            lastIdx = hash[lastIdx];
        }
        result.add(nums[lastIdx]); // add the root element
        Collections.reverse(result);

        return result;

    }
}
```

Binary Trees:

Traversal in Binary Trees:

- BFS - Breadth First Search / Level Wise Traversal
- DFS - Depth First Search
  - Inorder → `Left-Root-Right`
  - Preorder → `Root-Left-Right`
  - Post-order → `Left-Right-Root`

Preorder Traversal

- Recursive
  TC - O(N)
  SC- O(N) Stack Space

```java
import java.util.* ;
import java.io.*;

/*********************************************************
Following is the BinaryTreeNode class structure

class BinaryTreeNode<T> {
    T data;
    BinaryTreeNode<T> left;
    BinaryTreeNode<T> right;

    public BinaryTreeNode(T data) {
        this.data = data;
    }
}
*********************************************************/
import java.util.ArrayList;

public class Solution {
    public static ArrayList<Integer> preorderTraversal(BinaryTreeNode<Integer> root) {
        // Write your code here
        ArrayList<Integer> list = new ArrayList<>();
        findSolution(root, list);
```